

Отчёт по лабораторной работе №25-26

по курсу «Алгоритмы и структуры данных».

Выполнил студент группы М8О-114БВ-25 Миронова Татьяна Михайловна № по списку 12.

Контакты: tatirona@gmail.com

Работа выполнена: «27» мая 2026 г.

Преподаватель: каф.806 Сысоев Максим Алексеевич

Входной контроль знаний с оценкой: _____

Отчет сдан «27» мая 2026 г.

Итоговая оценка: _____

Подпись преподавателя: _____

1. **Тема:** Автоматизация сборки программ модульной структуры на языке Си с использованием утилиты make + Абстрактные типы данных. Рекурсия. Модульное программирование на языке Си.

2. **Цель работы:** Составить и отладить модуль определений и модуль реализации по заданной схеме модуля определений для абстрактного (пользовательского) типа данных (стека, очереди, списка или дека). Составить программный модуль, сортирующий экземпляр указанного абстрактного типа данных заданным методом, используя только операции, импортированные из модуля UUDT. Составить Makefile для данной модульной программы.

3. **Задание:** 2. Очередь. Процедура: Вставка элемента в стек, дек, список или очередь, упорядоченные по возрастанию, с сохранением порядка. Метод: сортировка простой вставкой.

4. **Оборудование:** *Оборудование ПЭВМ студента, если использовалось:*

Процессор: Intel(R) Pentium(R) CPU 6405U @ 2.40GHz 2.40 GHz, ОП 8 ГБ, SSD 256 ГБ

5. **Программное обеспечение:** *Программное обеспечение ЭВМ студента, если использовалось:*

Операционная система семейства Unix, наименование Ubuntu, версия 24.04 LTS, интерпретатор команд bash версия 5.2.21.

Система программирования: gcc (Ubuntu 11.4.0-1ubuntu1~22.04.2) версия 11.4 Редактор текстов: VS code версия: 1.107.1.

Язык программирования: C.

6. **Идея, метод, алгоритм решения задачи** [в формах: словесной, псевдокода, графической (блок-схема, диаграмма, рисунок, таблица) или формальные спецификации с пред- и постусловиями]:

Словесное описание идеи и метода

В рамках данных лабораторных работ реализуется абстрактный тип данных (АТД) **Очередь** на базе динамического массива, а также операции над ним.

Процедура №2 (Вставка с сохранением порядка):

Для вставки элемента в очередь, упорядоченную по возрастанию ключей, применяется метод линейного поиска позиции. Алгоритм просматривает элементы от начала очереди к концу. Позицией вставки является индекс первого элемента, чей ключ строго больше или равен ключу вставляемого элемента. После определения позиции все последующие элементы сдвигаются вправо на одну позицию (в сторону хвоста), освобождая место для нового элемента. Если такого элемента не найдено, элемент добавляется в самый конец очереди. При необходимости динамическая память автоматически перевыделяется с удвоением емкости.

Метод №2 (Сортировка простой вставкой, рекурсивная версия):

Алгоритм рекурсивной сортировки вставками основан на декомпозиции задачи.

1. Базовый случай: Очередь, содержащая 0 или 1 элемент, априори считается отсортированной, рекурсивный вызов завершается.
2. Шаг рекурсии: Из очереди временно извлекается последний элемент (размер очереди уменьшается на единицу). Затем функция рекурсивно вызывает саму себя для оставшейся «усеченной» очереди.

После того как рекурсивный вызов возвращает управление (и левая часть очереди гарантированно отсортирована), извлеченный ранее элемент вставляется обратно с помощью Процедуры №2, которая находит для него корректное место с сохранением упорядоченности.

Формальные спецификации с пред- и постусловиями

Процедура queue_insert(q, d):

- Предусловие (Pre): Очередь \$q\$ инициализирована и упорядочена по возрастанию ключей: $\forall i \in [0, q.size - 2]: q.data[i].key \leq q.data[i+1].key$.
- Постусловие (Post): Элемент \$d\$ добавлен в очередь \$q\$. Размер очереди увеличился на 1 ($q.size_{new} = q.size_{old} + 1$). Очередь сохранила упорядоченность по возрастанию ключей: $\forall i \in [0, q.size_{new} - 2]: q.data[i].key \leq q.data[i+1].key$.

Метод insertion_sort(q):

- Предусловие (Pre): Очередь \$q\$ инициализирована, содержит \$n\$ элементов ($q.size = n, n \geq 0$). Порядок элементов произвольный.
- Постусловие (Post): Размер очереди не изменился ($q.size = n$). Элементы очереди перераспределены таким образом, что их ключи упорядочены по возрастанию: $\forall i \in [0, q.size - 2]: q.data[i].key \leq q.data[i+1].key$.

Псевдокод:

ФУНКЦИЯ queue_insert(Переменная q: Очередь, d: Элемент)

ВЫЗВАТЬ ensure_capacity(q) // Проверка и расширение памяти при необходимости

pos = 0

// Поиск позиции для вставки

ПОКА pos < q.size И q.data[pos].key < d.key ЦИКЛ

pos = pos + 1

КОНЕЦ ЦИКЛА

// Сдвиг элементов вправо для освобождения места

ДЛЯ i ОТ q.size ВНИЗ ДО pos + 1 ЦИКЛ

q.data[i] = q.data[i - 1]

КОНЕЦ ЦИКЛА

// Вставка элемента и обновление размера

q.data[pos] = d

q.size = q.size + 1

КОНЕЦ ФУНКЦИИ

ФУНКЦИЯ insertion_sort(Переменная q: Очередь)

// Базовый случай рекурсии

ЕСЛИ q.size <= 1 ТОГДА

ВЫХОД ИЗ ФУНКЦИИ

КОНЕЦ ЕСЛИ

// Извлечение последнего элемента

last_element = q.data[q.size - 1]

q.size = q.size - 1

// Рекурсивный вызов для сортировки под-очереди

ВЫЗВАТЬ insertion_sort(q)

// Вставка извлеченного элемента в отсортированную структуру

ВЫЗВАТЬ queue_insert(q, last_element)

КОНЕЦ ФУНКЦИИ

7. Сценарий выполнения работы (план работы, первоначальный текст программы в черновике (можно на отдельном листе) и тесты либо соображения по тестированию).

План работы:

1. Разработка структуры данных data_type (с полями key и value) и управляющей структуры queue с указателем на динамический массив, переменными размера (size) и емкости (capacity).
2. Проектирование и написание заголовочного файла queue.h, фиксирующего интерфейс АТД Очередь.

3. Реализация базовых функций управления памятью (queue_create, ensure_capacity) и стандартных операций очереди (queue_push_back, queue_pop_front и др.) в файле queue.c.
4. Реализация целевой Процедуры №2 (queue_insert) и рекурсивного Метод №2 (insertion_sort).
5. Написание демонстрационного модуля main.c для генерации потока проверочных данных.
6. Написание Makefile для автоматизации раздельной компиляции модулей и их последующей компоновки в исполняемый файл prog.
7. Проведение комплексного тестирования, исправление логических ошибок сдвига индексов при добавлении/удалении элементов.

Соображения по тестированию:

Для подтверждения корректности работы АТД и алгоритмов необходимо сформировать тестовые наборы, покрывающие следующие критические ситуации:

- Функционирование базовых механизмов очереди: добавление элементов в пустую очередь, последовательное заполнение, корректность удаления из головы (FIFO) и хвоста.
- Поведение алгоритма сортировки на граничных случаях: пустая очередь (размер 0) и очередь из одного элемента. Алгоритм не должен уходить в бесконечную рекурсию или вызывать ошибки сегментации.
- Поведение сортировки на худшем (элементы отсортированы в обратном порядке) и лучшем (элементы уже отсортированы) наборах данных.
- Точечная проверка Процедуры №2: вставка нового элемента в начало упорядоченной очереди, в ее середину и в самый конец.

Пункты 1-7 отчета составляются строго до начала лабораторной работы.

Допущен к выполнению работы. Подпись преподавателя _____

8. Распечатка протокола (подклеить листинг окончательного варианта программы с тестовыми примерами, подписанный преподавателем).

Листинг окончательного варианта программы (Bash):

1. Файл Makefile

Makefile для лабораторных работ №25 и №26

Вариант: Очередь + Сортировка простыми вставками

Компилятор и флаги

CC = gcc

CFLAGS = -Wall -g -std=c99

LDFLAGS =

Целевой исполняемый файл

TARGET = prog

Объектные файлы

OBJS = main.o queue.o

Правило по умолчанию

all: \$(TARGET)

Компоновка

\$(TARGET): \$(OBJS)

\$(CC) \$(LDFLAGS) -o \$(TARGET) \$(OBJS)

Компиляция main.o

main.o: main.c queue.h

\$(CC) \$(CFLAGS) -c main.c

Компиляция queue.o

queue.o: queue.c queue.h

\$(CC) \$(CFLAGS) -c queue.c

Очистка временных файлов

clean:

rm -f \$(OBJS) \$(TARGET)

Полная пересборка проекта

rebuild: clean all

.PHONY: all clean rebuild

2. Файл queue.h

```
#ifndef QUEUE_H
#define QUEUE_H

#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>

// Типы для ключа и значения
typedef int key_type;
typedef int value_type;

// Структура элемента данных
typedef struct {
    key_type key;
    value_type value;
} data_type;

// Структура очереди на базе динамического массива
typedef struct {
    data_type* data; // Массив элементов
    size_t size; // Текущее количество элементов
    size_t capacity; // Выделенная память
} queue;

// Базовые операции (Интерфейс АТД)
void queue_create(queue* q);
bool queue_is_empty(const queue* q);
void queue_push_front(queue* q, data_type d);
void queue_push_back(queue* q, data_type d);
void queue_pop_front(queue* q);
void queue_pop_back(queue* q);
void queue_print(const queue* q);
size_t queue_size(const queue* q);
void queue_erase(queue* q, const key_type k);

// Процедура №2: Вставка элемента в упорядоченную по возрастанию очередь с сохранением
// порядка
void queue_insert(queue* q, const data_type d);

// Метод №2: Сортировка простой вставкой (рекурсивная реализация)
void insertion_sort(queue* q);

#endif
```

3. Файл queue.c

```
#include "queue.h"

#define INIT_CAPACITY 10

// Создание очереди
void queue_create(queue* q) {
    q->data = (data_type*)malloc(INIT_CAPACITY * sizeof(data_type));
    q->size = 0;
    q->capacity = INIT_CAPACITY;
}

// Проверка на пустоту
bool queue_is_empty(const queue* q) {
    return q->size == 0;
}

// Вспомогательная процедура автоматического расширения массива
static void ensure_capacity(queue* q) {
    if (q->size >= q->capacity) {
        q->capacity *= 2;
    }
}
```

```

        q->data = (data_type*)realloc(q->data, q->capacity * sizeof(data_type));
    }
}

// Добавление в начало (сдвиг элементов вправо)
void queue_push_front(queue* q, data_type d) {
    ensure_capacity(q);
    for (size_t i = q->size; i > 0; i--) {
        q->data[i] = q->data[i - 1];
    }
    q->data[0] = d;
    q->size++;
}

// Добавление в конец
void queue_push_back(queue* q, data_type d) {
    ensure_capacity(q);
    q->data[q->size++] = d;
}

// Удаление из начала (сдвиг элементов влево)
void queue_pop_front(queue* q) {
    if (q->size == 0) return;
    for (size_t i = 0; i < q->size - 1; i++) {
        q->data[i] = q->data[i + 1];
    }
    q->size--;
}

// Удаление из конца
void queue_pop_back(queue* q) {
    if (q->size > 0) {
        q->size--;
    }
}

// Вывод содержимого очереди в стандартный поток
void queue_print(const queue* q) {
    printf("[");
    for (size_t i = 0; i < q->size; i++) {
        printf("(%d,%d)", q->data[i].key, q->data[i].value);
        if (i < q->size - 1) printf(" ");
    }
    printf("]\n");
}

// Получение текущего размера очереди
size_t queue_size(const queue* q) {
    return q->size;
}

// Удаление элемента по ключу
void queue_erase(queue* q, const key_type k) {
    size_t pos = 0;
    while (pos < q->size && q->data[pos].key != k) {
        pos++;
    }
    if (pos < q->size) {
        for (size_t i = pos; i < q->size - 1; i++) {
            q->data[i] = q->data[i + 1];
        }
        q->size--;
    }
}

// Процедура №2: Вставка элемента в упорядоченную по возрастанию структуру
void queue_insert(queue* q, const data_type d) {

```

```

ensure_capacity(q);
size_t pos = 0;
// Ищем позицию для вставки (первый элемент, чей ключ больше ключа d)
while (pos < q->size && q->data[pos].key < d.key) {
    pos++;
}
// Освобождаем место под новый элемент, сдвигая хвост вправо
for (size_t i = q->size; i > pos; i--) {
    q->data[i] = q->data[i - 1];
}
q->data[pos] = d;
q->size++;
}

// Метод №2: Рекурсивная сортировка простыми вставками
void insertion_sort(queue* q) {
    // Базовый случай рекурсии: очередь из 0 или 1 элемента уже отсортирована
    if (q->size <= 1) return;

    // Шаг 1: "Вытаскиваем" последний элемент из очереди
    data_type last_element = q->data[q->size - 1];
    q->size--; // Временно уменьшаем размер (аналог pop_back)

    // Шаг 2: Рекурсивно сортируем оставшуюся усеченную очередь
    insertion_sort(q);

    // Шаг 3: Вставляем извлеченный элемент на его законное место с помощью Процедуры №2
    queue_insert(q, last_element);
}

```

4. Файл main.c

```

#include "queue.h"
#include <stdio.h>

int main() {
    queue myQueue;
    queue_create(&myQueue);

    printf("=== Демонстрация работы АТД Очередь (Вариант 2) ===\n\n");

    // 1. Наполнение элементами вразнобой
    printf("1. Наполняем очередь элементами:\n");
    data_type items[] = { {30, 300}, {10, 100}, {50, 500}, {20, 200}, {40, 400} };
    for (int i = 0; i < 5; i++) {
        queue_back(&myQueue, items[i]); // имитируем стандартный push_back
        // Но используем имя из нашего интерфейса:
        queue_push_back(&myQueue, items[i]);
        printf("    Добавлен элемент с ключом: %d\n", items[i].key);
    }

    printf("\n2. Текущее состояние очереди: ");
    queue_print(&myQueue);

    // 2. Тестирование сортировки
    printf("\n3. Запуск рекурсивной сортировки простыми вставками:\n");
    insertion_sort(&myQueue);
    printf("    Очередь после сортировки: ");
    queue_print(&myQueue);

    // 3. Тестирование Процедуры №2 (Вставка в упорядоченную структуру)
    printf("\n4. Тестирование Процедуры №2 (Вставка элемента {25, 250} с сохранением порядка):\n");
    data_type newItem = {25, 250};
    queue_insert(&myQueue, newItem);
    printf("    Результат вставки: ");
    queue_print(&myQueue);
}

```

```

// 4. Демонстрация удаления и проверки свойств
printf("\n5. Извлечение элементов и проверка АД:\n");
printf(" Текущий размер: %zu\n", queue_size(&myQueue));

printf(" Удаляем первый элемент из головы (pop_front): ");
queue_pop_front(&myQueue);
queue_print(&myQueue);

printf(" Удаляем элемент с ключом 40 (queue_erase): ");
queue_erase(&myQueue, 40);
queue_print(&myQueue);

printf("\n=== Тестирование успешно завершено ===\n");

// Освобождение ресурсов
free(myQueue.data);
return 0;
}

```

Распечатка протокола:

\$ make

gcc -Wall -g -std=c99 -c main.c

gcc -Wall -g -std=c99 -c queue.c

gcc -o prog main.o queue.o

\$./prog

=== Демонстрация работы АД Очередь (Вариант 2) ===

1. Наполняем очередь элементами:

Добавлен элемент с ключом: 30

Добавлен элемент с ключом: 10

Добавлен элемент с ключом: 50

Добавлен элемент с ключом: 20

Добавлен элемент с ключом: 40

2. Текущее состояние очереди: [(30,300) (10,100) (50,500) (20,200) (40,400)]

3. Запуск рекурсивной сортировки простыми вставками:

Очередь после сортировки: [(10,100) (20,200) (30,300) (40,400) (50,500)]

4. Тестирование Процедуры №2 (Вставка элемента {25, 250} с сохранением порядка):

Результат вставки: [(10,100) (20,200) (25,250) (30,300) (40,400) (50,500)]

5. Извлечение элементов и проверка АД:

Текущий размер: 6

Удаляем первый элемент из головы (pop_front): [(20,200) (25,250) (30,300) (40,400) (50,500)]

Удаляем элемент с ключом 40 (queue_erase): [(20,200) (25,250) (30,300) (50,500)]

9. **Дневник отладки** должен содержать дату и время сеансов отладки и основные события (ошибки в сценарии и программе, нестандартные ситуации) и краткие комментарии к ним. В дневнике отладки приводятся сведения об использовании других ЭВМ, существенном участии преподавателя и других лиц в написании и отладке программы.

№	Лаб. или дом.	Дата	Время	Событие	Действие по исправлению	Примечание

10. **Замечания автора** по существу работы: замечания отсутствуют.

11. **Выводы:** В ходе лабораторной работы разработаны два скрипта (Bash и Python) для создания копий файла с автоматической генерацией имён путём добавления последовательных букв или цифр. Реализована корректная обработка переноса разрядов в буквенном режиме (после 'z' идёт 'za'). Скрипты включают проверку входных данных и обработку ошибок. Работа позволила закрепить навыки файлового ввода-вывода, обработки аргументов командной строки и строковых операций. Оба скрипта успешно протестированы в Ubuntu.

Подпись студента _____